Pacman Final Report Group N

ZEMYNA KULIKAUSKAITE, Trinity College Dublin
SHEENA O'REILLY, Trinity College Dublin
THOMAS MORONEY, Trinity College Dublin
YONGJIA LI, Trinity College Dublin
WEN GENG LIN, Trinity College Dublin

1 INTRODUCTION

This report presents the design and implementation of a distributed multiplayer Pac-Man game, that runs on a localhost web browser. The game uses technologies such as WebSockets, Redis, and MongoDB to provide a seamless multiplayer experience, ensuring real-time state synchronisation across multiple devices. The system's architecture supports scalability, fault tolerance, and performance optimisation, allowing players to connect and interact in a dynamic and persistent maze environment. The report outlines the functional and non-functional requirements, technical specifications, system architecture, implementation details, and testing.

2 REQUIREMENTS

2.1 Service Description

The application is a multiplayer Pac-Man game designed to be played across multiple devices. Our distributed system is currently simulated locally using Docker containers, with players joining via separate browser instances. All players share the same maze environment, interacting with the same ghosts and collecting the same dots.

The game features an endless maze mechanic, when a player exits through a door, they are transported into a new maze instance, allowing continuous gameplay. The system is distributed across multiple servers, with Redis used to maintain consistency of game state across instances.

A scoreboard tracks individual player scores in real-time, which is managed using a central MongoDB database. This architecture allows the game to scale to multiple users while preserving state synchronisation and score accuracy across all clients.

2.2 Functional Requirements

The functional requirements section defines the system's specific features and behaviour.

- **Player Requirements**
  - Players can join the game through individual browser instances.
  - Each player can see other active players that are in the same maze in real time, even if they are connected to a different server.
  - Player and ghost movements are synchronised across all connected clients.
  - Dot positions are consistent and synchronised for all players.
  - Ghosts interact consistently with all players, regardless of client.
  - If multiple players collide with the same ghost simultaneously, all involved players are considered dead.
  - Players can collect dots and accumulate points.
  - Players have three lives that decrease when colliding with ghosts.
  - Players can eat power ups and eat ghosts.
  - Players have three lives and then they are presented with a game over screen.
- **Maze Requirements**
  - When a player exits through a door, they are transitioned into a new maze.
  - Each maze has a unique visual texture, allowing players to identify when they have entered a different maze.
- **Ghost and Dropper Requirements**
  - Ghosts remain confined to the maze in which they originally spawned.
  - When a ghost is eaten, it respawns after a short delay.
  - Ghosts respawn with one less life if eaten, they do not respawn if they run out of lives, if all the ghosts die, all 4 respawn.
  - Droppers move randomly and dynamically place dots in areas where none currently exists.
- **Additional Requirements**
  - A scoreboard displays the top 10 players with the highest scores.
  - The scoreboard updates in real time as scores change.
  - The server and maze instance associated with a player is visibly displayed to them.
  - The number of players currently present in the same maze is shown to each player.
  - Servers dynamically multiply to handle load.

2.3 Non-Functional Requirements

The non-functional requirements define how a system performs a function rather than what functions it performs. They describe the quality, performance, and constraints of the system.

- **Scalability**
  - The system supports multiple game server instances, each running on a separate port.
  - Load balancing is implemented to distribute player connections across server instances.
  - We use sharding to dynamically assigned mazes to servers based on player distribution.

  - Additional server instances can be added without requiring code modifications.
  - The system is deployed in a regional architecture, with one server cluster per region (e.g., EU, Asia). Each cluster has its own Redis database instance and we use replication to synchronise state across regions.
- **Performance**
  - Ghost movement updates are optimised to occur 20 times per second.
  - Real-time communication is handled via a WebSocket protocol that transmits only essential data.
  - Network latency is minimized through movement interpolation on the client side.
  - Static assets such as images and map files are cached to reduce loading times and bandwidth consumption.
- **Availability**
  - If a player's connection to a server is lost, their current state (position, direction, etc.) is preserved.
  - The client automatically attempts to reconnect to the next available server instance within the same region.
  - On reconnection, the player's game state is synchronised with the game server to maintain continuity.
- **Data Management**
  - **Redis:** Used for real-time game state management across servers. It tracks transient data including player positions, ghost positions, dropper activity, consumed dots, maze metadata, and power dots. Expiry times are set for certain keys (e.g., 1 hour for players, 24 hours for dots) to manage memory usage.
  - **MongoDB:** Used for long-term storage of persistent data such as player scores, game history, and static configuration data. MongoDB ensures data durability and historical recordkeeping beyond individual game sessions.

3 SPECIFICATION

3.1 System Overview

The system is a distributed multiplayer Pac-Man game designed to run on web browsers. Players connect to game servers via WebSocket, enabling real-time communication for player actions and game state synchronisation. Redis is used for managing real-time game data, such as player positions, ghost states, and maze updates. MongoDB handles persistent storage, including player scores, profiles, and game history.

3.2 Components

Description of the key components:

- **Client**: Is responsible for rendering the game and managing player interactions in real time. It handles the display of various game components, such as the maze layout, player positions, and ghosts. The client also ensures synchronisation with the server by managing WebSocket connections and coordinating multiplayer interactions. Additionally, the client dynamically updates the leaderboard and tracks player actions, like movement and consumption of dots.
- **Backend**: Is responsible for game state synchronisation, persistent storage, player connections, Websocket connections, Redis integration, and MongoDB for storage. Three Docker-based servers manage player requests and distribute game state management across a scalable, distributed setup.
- **Redis**: Stores real-time data (positions, maze data), auto-expiry for transient values.
- **MongoDB**: Stores scores, history.

3.3 Fault Tolerance and Error Handling

Describes what happens when something goes wrong:

- If a player disconnects, their state is stored in Redis and restored on reconnect.
- If a server fails, players are redirected to another server in the cluster.
- If Redis crashes, real-time data is lost, but persistent data in MongoDB remains.

3.4 Limitations

Some features are not fully implemented or partially working:

- Currently there is no implementation of dynamically multiplying servers to handle load. We depend on the sharding of the infinite mazes to spread out the load across servers, but this will not hold up if a large amount of players decide to meet up in a single maze. Replication can be implemented to fix this.
- Currently there is only one server cluster (one region), but this is currently being implemented, and with replication of Redis and hopefully will be implemented before the demonstration.
- It is ran locally and therefore two players can not move at the same time as the same movement keys on the keyboard are used for all game instances i.e. you would need to have multiple keyboards (or multiple devices) to control multiple players at once.
- All the ghosts currently respawn if one ghost loses all three lives rather than waiting until all four are dead, but this is currently being implemented and hopefully will be implemented before the demonstration.

3.5 Distributed Techniques

Process Replication : Multiple game servers (e.g., game-server-0, game-server-1) each handle a different part of the maze. This helps balance the load and makes the system more fault-tolerant.

- **Degree of Isolation**: Each server manages a different maze section, so failures are isolated and don't impact the whole system.
- **Degree of Replication**: Each server replicates its local game state in real time using Redis, but full replication across all servers is not used.

- **Level of Replica Consistency**: Eventual consistency is acceptable since each server manages separate parts of the maze.
- **Assumption about Fault Tolerance**: The system redirects players from failed servers to available ones to ensure continued gameplay.
- **Assumption about failure model**: It assumes temporary server failures that can be handled by other servers taking over the load.

Partitioning : The maze is split into sections, with each server managing a number of mazes. Players are assigned to servers based on where they are in the maze, and new servers can be added to support more players or mazes. This is the formula we use for sharding the mazes across servers,

$$serverId = (mazeX + mazeY) \mod totalServers$$

.

- **Degree of Isolation**: Partitioning gives each server control over a specific maze section, limiting the impact of failures.
- **Assumption about Fault Tolerance**: Players can be redirected if a server fails, ensuring other sections remain unaffected.
- **Assumption about failure model**: Server-level failures are expected and handled by re-routing players to a different sever and maze without disrupting gameplay.

Caching : Redis stores game data such as player, ghost and dot positions, in memory for quick read and write operations. This reduces the reliance on the slower MongoDB database which is used to store less real-time critical data such as the leader-board.

- **Degree of Isolation**: Caching is isolated per server cluster, allowing independent in-memory management of game data.
- **Degree of Replication**: Redis replicates data in real time across clusters to keep memory state consistent.
- **Level of Replica Consistency**: Redis uses eventual consistency, which is acceptable as this is a game.
- **Assumption Fault Tolerance**: Redis failures may result in data loss.
- **Assumption about failure model**: Redis may crash.

Load Balancing : Players are distributed across different servers to make sure no single server gets overloaded.

- **Degree of Isolation**: Load balancing spreads players across servers, ensuring that one server's failure doesn't affect the entire system.
- **Assumption about Fault Tolerance**: Overloaded servers trigger new servers to spin up which will and redistribution of the mazes across the servers, as well as the players within them, helping the system handle high loads gracefully.
- **Assumption about failure model**: The system assumes overloaded servers may fail and adjusts assignments to maintain gameplay.

Checkpointing/Message Logging : Redis keeps the game state synced in real-time. The NetworkManager logs important game events to help recover the game state if anything goes wrong.

- **Degree of Isolation**: Each server maintains and logs its own state, allowing isolated recovery without disrupting the whole system.
- **Degree of Replication**: Game state is replicated in Redis for real-time use.
- **Level of Replica Consistency**: Checkpointing supports strong consistency by enabling recovery of accurate game state from backups.
- **Assumption about failure model**: The system expects possible server or Redis failures and uses persistent logging to recover lost state.

Replication : Redis replicates data across server clusters to make sure players can play together in real-time, no matter where they are in the world.

- **Degree of Isolation**: Redis clusters replicate data internally, isolating failure impact and ensuring local redundancy.
- **Assumption about Fault Tolerance**: Redis ensures data availability even if one replica fails, maintaining seamless game continuity.
- **Assumption about failure model**: The system anticipates server or network failures and relies on Redis replication to preserve availability and data integrity.

3.6 API Descriptions

WebSocket API : This enables real-time communication between the client and server for syncing game state and player actions. It allows the server to send updates to the client and lets clients send their movements back to the server.

Canvas API : Used on the client-side to render game elements like walls and dots on the maze. It provides functions for drawing images and shapes on the canvas to represent the game visually.

MongoDB API : Used to save player data, scores, and game state. MongoDB handles long-term storage, like player profiles and game history, and Mongoose makes it easier to query and save this data.

Redis API : Handles real-time server-to-server communication by publishing and subscribing to game events. Redis ensures that game state updates, like player positions and maze changes, stay synced across multiple servers.

4 ARCHITECTURE

4.1 Overview

The system follows a distributed architecture, separating client-side rendering, server-side logic, and backend storage. It consists of game clients, game server instances, Redis for real-time state synchronisation, and MongoDB for long-term persistence.

4.2 Technical Stack

Frontend: React-based frontend with WebSocket communication
Backend: Node.js servers with Express and WebSocket

Database: MongoDB for persistence, Redis for caching
Containerisation: Docker-based deployment using docker-compose

4.3 Technical Architecture

This section outlines the components of the application and their functionalities.

4.3.1 Client-Side Components. :

The client-side handles rendering the game and managing player interactions.

Distributed Visualisation : Renders the distributed table, showing the player id, maze coordinates, server, player count, and connection status. It retrieves the information from the server.

Ghost Manager : Sets the ghosts starting position on the client side along with the ghosts speed. It also keeps track of the moving direction, controls ghost lives, sets how often the ghost direction is changed, and manages the blinking effect when the ghost is about to expire.

Game Logic : Integrates various game components, including Pacman, ghosts, networking, distributed visualisation, and leaderboard functionality.

Leaderboard UI : Creates and manages a dynamic, auto-refreshing leader board UI for a game, fetching and displaying player scores in real-time from a server.

Maze Manager : Controls the maze layout and structure, entity placement, maze transitions and wall image toggling, dot and power-up consumption tracking, dropper management and effects, maze coordinates and server switching, and canvas drawing for all game elements.

Moving Direction : Maps four movement directions (right, down, left, up) to their respective numeric values (0 to 3) and exports it for use in other parts of a program.

Network Manager : Controls managing WebSocket connections, sending/receiving player movements and actions, synchronising game state, handling maze transitions between servers, managing player connections and disconnections, and coordinating multi-player interactions.

Player Manager : Controls the player, handling movement, animation, interactions with the maze (dots, power dots, and ghosts), player lives, and networked game state updates for multiplayer gameplay.

4.3.2 Backend Components. :

The server-side is responsible for managing game state and ensuring synchronisation between players.

Game Server : manages the WebSocket server setup and handles player connections. It integrates Redis game state persistence across servers and maintain state in the case of server crashes. MongoDB is used to store the game's scoreboard as this isn't updated as often. The server coordinates maze management, player state synchronisation, and ghost behaviour. It also ensures game state persistence and recovery, while handling the overall multiplayer game logic.

Server : Three local Docker-based servers handle player requests and game state management in a distributed setup.

4.3.3 Shared Services. These services provide real-time and persistent data storage and synchronisation.

MongoDB : Used for long term storage, it stores, player profiles, player scores, game history and persistent game data. MongoDB is used for long term data.

Redis : Manages and tracks game states, such as player positions, Ghost positions, Dropper positions, consumed dots, maze metadata and power dots, across multiple servers. Redis will expire after set times (e.g. 1 hour for players, 24 hours for dots)and is used for real-time updates.

4.4 Architectural Diagram

Clients connect via WebSocket to the game server, with bi-directional communication for real-time updates. Game servers read/write player positions, ghost locations, maze transitions to Redis, and scores/leaderboards to MongoDB, as illustrated in Figure 1. Figure 2 shows the scaling of server clusters by region (EU, Asia) with replicated Redis databases to enable cross-region gameplay.

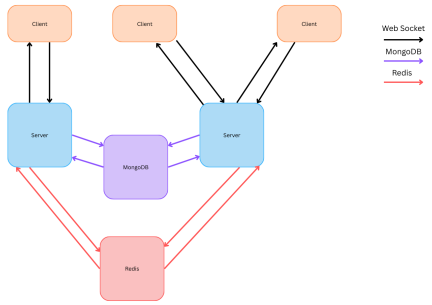

Fig. 1. Diagram of the Pacman client-server architecture.

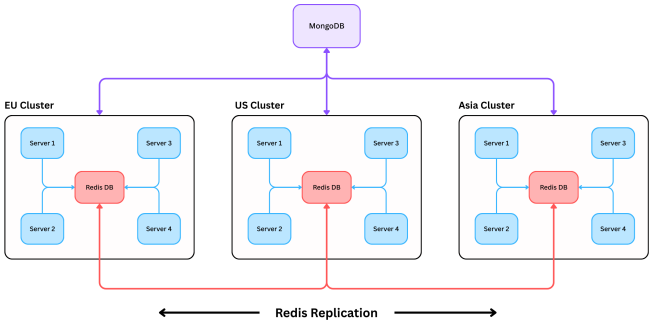

Fig. 2. Diagram of the Pacman backend architecture.

5 IMPLEMENTATION

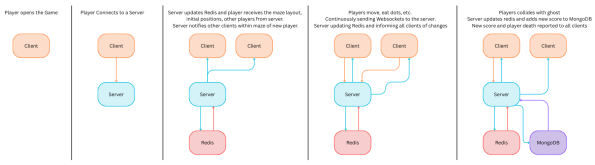

Fig. 3. Diagram of the overall Operation.

Overall Operation : Figure 3 shows the general flow of gameplay.

- Client connects to game server via WebSocket.
- Server assigns player to a maze (Redis handles which maze is active).
- Game loop updates player/ghost positions and syncs with Redis 20x/sec.
- State changes trigger updates to the Redis database.

5.1 Flow Diagrams

This section outlines the key flow diagrams.

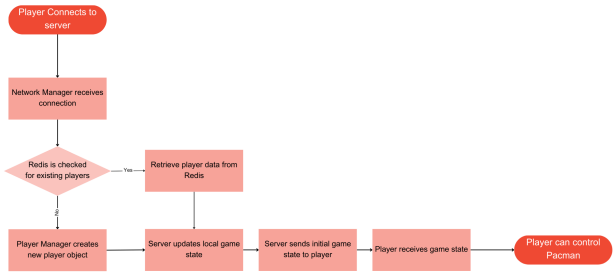

Fig. 4. Player joining game flow.

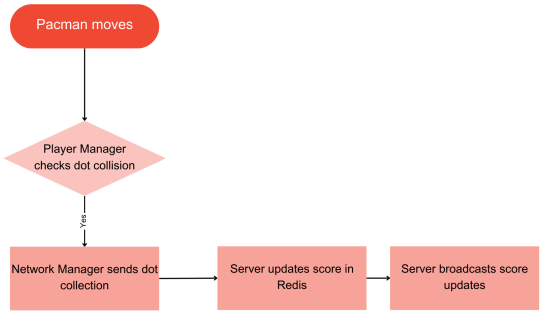

Fig. 5. Player collecting dot and score updating flow.

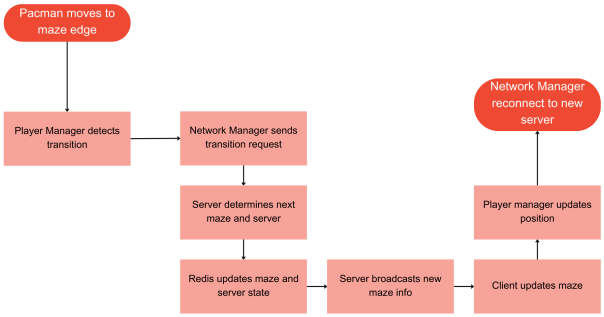

Fig. 6. Maze Transition Flow.

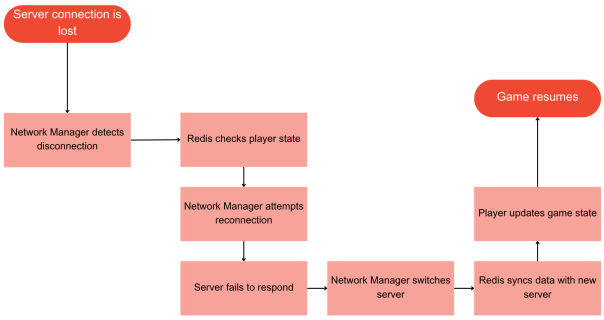

Fig. 7. Server Failure and Player Reconnection Flow.

5.2 Failure Handling

- Player Disconnect
  - Redis stores position, direction, and score.
  - On reconnect, server assigns player to correct maze from Redis.
  - Game resumes seamlessly.

- Server Crash
  - Players move to another server. Redis ensures state is restored.
- Redis Failure
  - Real-time data is lost.
  - Game state cannot be fully restored (acknowledged in limitations)
  - MongoDB still stores scores.
- Network Failure
  - Since it is ran locally nothing changes. The followng points descibe what would happen if it was deployed.
  - Players temporarily freeze.
  - Upon reconnection, sync from Redis

5.3 Tools and Libraries

- Node.js, Express, WebSockets. These technologies form the core of the backend server infrastructure:
  - Node.js provides the runtime environment for running JavaScript on the server side. It's event-driven and non-blocking, making it ideal for real-time multiplayer games.
  - Express is a lightweight web application framework for Node.js. It's used to define REST endpoints and serve static assets like the client-side game files.
  - WebSockets enable real-time, bi-directional communication between clients (players) and the game server. This allows player actions (like movement) and game events (like dot collection or ghost movement) to be synchronised instantly across all connected clients.
- Redis, MongoDB. These are the two main databases used in the system, each serving a different purpose:
  - Redis is an in-memory data store used for real-time, transient game data. It holds player positions, ghost states, dot placements, and maze metadata.
  - MongoDB is a NoSQL document database used for persistent storage. It stores long-term data such as player profiles, and scores.
- Game loop logic (e.g., setInterval at 50ms). The game uses a loop that updates the game state approximately every 50 milliseconds (which is 20 times per second). This is implemented using JavaScript's setInterval function. This loop is responsible for:
  - Updating player and ghost positions.
  - Checking for collisions (e.g., with dots or ghosts).
  - Broadcasting updated state to all connected players.

  A 50ms interval provides a smooth gameplay experience while keeping CPU and network usage manageable.

5.4 API Examples

Listing 1. WebSocket API

```
class NetworkManager {
  constructor() {
    this.socket = new WebSocket('ws://game-server
        ');

    // Event handlers
    this.socket.onmessage = (event) => {
      const data = JSON.parse(event.data);
      this.handleMessage(data);
    };

    this.socket.onclose = () => {
      console.log('Connection closed');
    };
  }

  // Send movement
  sendMove(data) {
    this.socket.send(JSON.stringify({
      type: 'move',
      playerId: this.playerId,
      x: data.x,
      y: data.y
    }));
  }
}
```

The WebSocket API in Listing 1 creates a connection to a game server using the URL ws://game-server. It listens for incoming messages from the server and processes them by parsing the data and passing it to the handleMessage method. Additionally, the sendMove method sends a player's movement data (including x and y coordinates) to the server in a JSON format whenever called.

Listing 2. Canvas API

```
class MazeManager {
  constructor(canvas) {
    this.canvas = canvas;
    this.ctx = canvas.getContext('2d');
  }

  drawMaze() {
    // Draw walls
    this.walls.forEach(wall => {
      this.ctx.drawImage(this.wallImage, wall.x,
          wall.y);
    });

    // Draw dots
    this.dots.forEach(dot => {
      if (dot.active) {
        this.ctx.fillStyle = 'yellow';
        this.ctx.beginPath();
        this.ctx.arc(dot.x, dot.y, 3, 0, Math.PI *
            2);
        this.ctx.fill();
      }
    });
  }
}
```

In this Listing 2, the canvas api is used to render walls and active dots in a maze by drawing them on the 2D canvas context (this.ctx).

Listing 3. MongoDB API

```
const mongoose = require('mongoose');
```

```
const scoreSchema = new mongoose.Schema({
  playerId: { type: String, required: true },
  playerName: { type: String, default: 'Anonymous'
        },
  score: { type: Number, required: true },
  date: { type: Date, default: Date.now }
});

scoreSchema.index({ score: -1 });
const Score = mongoose.model('Score', scoreSchema)
    ;
module.exports = Score;
}
```

This Listing 3 defines a Mongoose schema for a Score model, which includes fields for playerId, playerName, score, and date. It also creates an index on the score field to allow efficient querying of scores in descending order and exports the Score model for use in other parts of the application.

Listing 4. Redis API

```
// Redis setup
const Redis = require('ioredis');
require('dotenv').config();

let redisClient = null;
const connectRedis = async () => {
  try {
    const redisUri = process.env.REDIS_URI || '
        redis://localhost:6379';
    redisClient = new Redis(redisUri);
    await redisClient.ping();
    console.log(`Redis Connected: ${redisUri}`);
    return redisClient;
  } catch (error) {
    console.error(`Error connecting to Redis: ${
        error.message}`);
    process.exit(1);
  }
};

// Redis in action

const mazeData = await redisService.
    getOrCreateMaze(x, y);
const consumedDots = await redisService.
    getConsumedDots(x, y);
const playersInMaze = await redisService.
    getPlayersInMaze(x, y);

await redisService.setPlayer(playerId, {
              id: playerId,
              x: data.x,
              y: data.y,
              direction: data.direction,
              score: player.score || 0,
              lives: player.lives || 3
            });
```

This Listing 4 sets up a Redis client using the ioredis library to connect to a Redis instance, either from a URI specified in the environment variables or by default to localhost:6379. It defines a connectRedis function to establish the connection and perform a ping check, and later uses Redis to store and retrieve game-related data such as maze data, consumed dots, and player information.

5.5 Game Screens

This section displays some of the key functionality in our application.

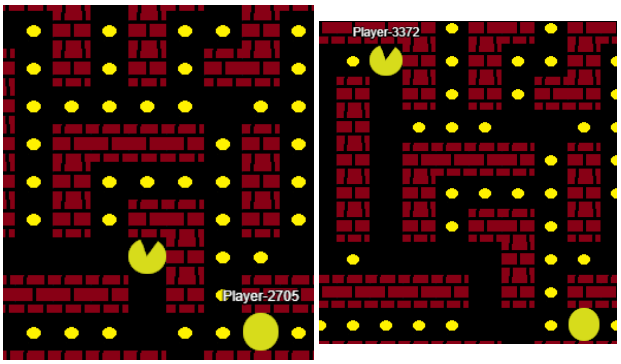

Fig. 8. Pacman eats dots and they disappear for all players

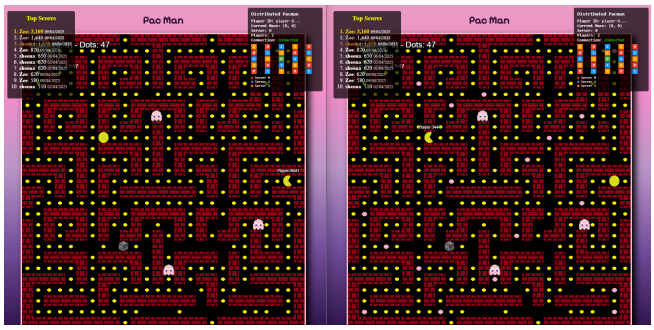

Fig. 9. The ghosts are synchronised across players within the maze.

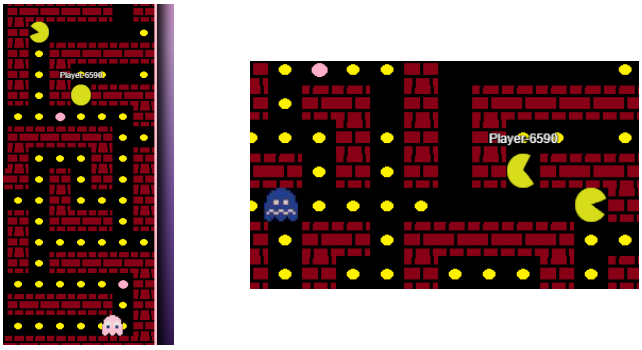

Fig. 10. When a player eats a power-up, the ghost becomes scared for that player.

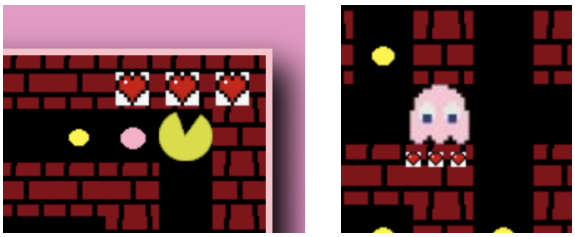

Fig. 11. Player and ghosts have 3 lives initially.

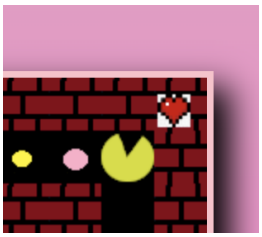

Fig. 12. Player loses life when it collides with a ghost.

Fig. 13. When a player has 0 lives left, the game will display a game over message.

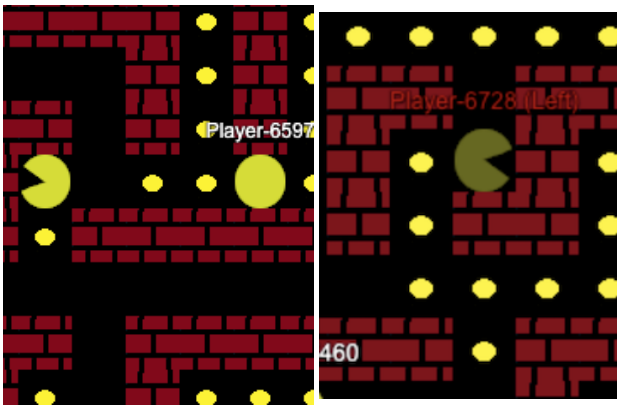

Fig. 14. Players can see each other, when playing the game and they can see when a player is leaving.

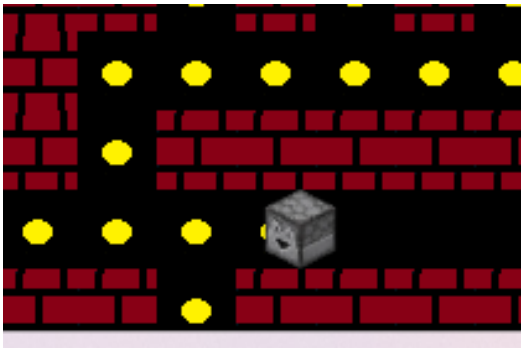

Fig. 15. The dropper will randomly replenish dots in areas where they have been eaten.

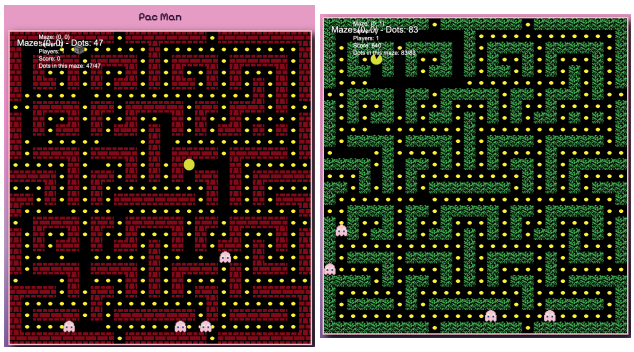

Fig. 16. Different mazes have different coloured walls to simply show the transition of mazes, maze(0,0) and maze(0, 1)

Fig. 17. Each player will see the distributed board which will show player id, current maze, the server they are on, the amount of players in the maze, the status of their connection, and the mazes around the current mazes with the servers they are on.

Fig. 18. Each player will see the leaderboard which will show the top 10 scores from themselves of different players. Since this is being ran locally this was two different instances of playing the game with two different IP Addresses.

6 TESTING

6.1 Functional Requirements

- We implemented a script that will automatically pull docker files and start docker images for the backend and leader board. We tested the script works for setting up the environment fully for easier deployment on all the machines. The Figure below shows that docker containers are pulled automatically by running the script.

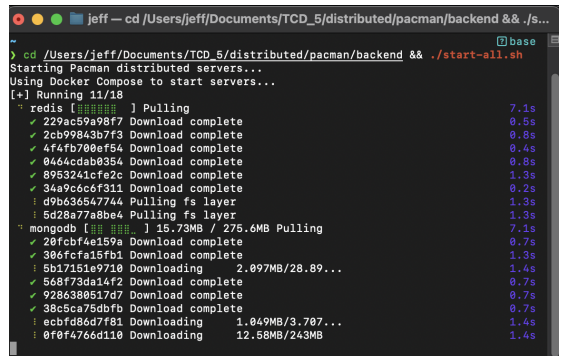

Fig. 19. auto-pull docker containers

- The single player is tested with eating dots and hit a ghost. When a dot is eaten, the dot should disappear and when the player hit a ghost, the player should die.
- The multiple player is tested by opening a new window. The other player in the original window should see the position of the other player is updated. When a player quites the game, the player should disappear from other's view.
- When the players moves out of the current maze, it will enter a new maze and the dot setup will be updated for a new maze. The look of the maze is updated as well.
- The leader board is tested by ending the game for both players and see if the high scores are updated. The leader board will only keep 10 highest scores.

Additional unit tests were defined to validate some other functional requirements. Using mocked data, these tests were created to confirm that core game features behave as expected.

The primary component under test within this section was the `MazeManager`, which orchestrates game state and transitions. All of these tests were implemented using Jest with fake timers to simulate time-based events (e.g., power dot expiration, ghost and dropper lifetimes).

Starting with player and enemy interactions, the tests confirm that Pacman loses a life upon colliding with a ghost unless a power dot is active. This is validated by simulating direct collisions in both powered and unpowered states, ensuring that the game's life management logic behaves correctly. Additionally, a separate test confirms that the game ends when Pacman's lives reach zero.

Dot collection and score tracking are also tested. Pacman's score correctly increases by 10 points when a standard dot is eaten and by 50 points when a power dot is consumed. Both types of dots are marked as consumed within the internal maze state, ensuring that the system accurately tracks which tiles have been cleared by the player.

Power dot effects are tested using a series of timing-based tests. These confirm that power dot activation triggers a temporary invincibility window, during which Pacman can eat ghosts without losing lives. The expiration of the power-up after its designated duration

is tested using mocked timers, and an additional test ensures that consuming a second power dot resets the timer appropriately.

Ghosts and droppers are tested for both lifecycle behaviour and grouping logic. Ghosts are created in batches of four, each with a shared batch ID, and can be removed in full by their batch identifier. The test suite confirms that ghosts expire after a fixed lifespan of 120 seconds, as designed. Similarly, droppers are created with specific coordinates and lifetimes, and are removed when they expire.

The test suite confirms that the maze transitions correctly when the player moves to a new sector, and that the grid resets to an empty state during the transition. It also verifies tile validation logic, confirming that Pacman cannot move through walls and that out-of-bounds movement is correctly restricted.

6.2 Non-Functional Requirements Testing Through Load Simulation

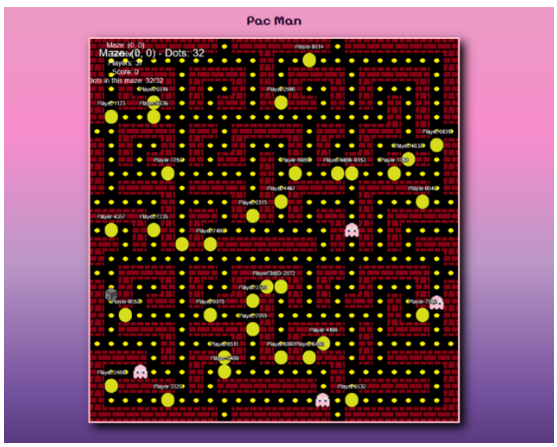

Fig. 20. Load Testing by Simulating Players

The load test simulates 5,000 concurrent players interacting with the game world, distributed geographically and logically across a 500 × 500 maze grid. The system is evaluated based on round-trip latency, CPU usage, server distribution, and message handling efficiency.

The simulation is implemented by creating thousands of WebSocket clients representing individual players. Each client connects to a server using the same load balancing technique as the game.

These clients track round-trip time (RTT) using timestamped messages.

The averaged RTT for a simulation with 5000 players was 342ms. This is a generally acceptable range for real-time games (ideally it would be <200ms). Players may perceive movement lag or delays in state updates, especially in the cases where they are transitioning between maze sectors or interacting with ghosts or power dots. The root cause for this is suspected to be the high client-serve message volume.

Server Distribution: For 5000 players across 3 servers, the distribution was as follows:

- Server 1 - 1502 Players (30%)
- Server 2 - 1471 Players (29%)
- Server 3 - 2027 Players (41%)

The distribution is reasonably balanced, with server 3 carrying a slightly higher proportion of players.

Maze Distribution for the top 5 mazes

- Maze (349, 271) - 3 Players
- Maze (199, 184) - 2 Players
- Maze (25, 469) - 2 Players
- Maze (197, 80) - 2 Players
- Maze (415, 227) - 2 Players

The players were randomised across mazes to showcase the large-scale functionality of the maze system.

7 SUMMARY

This project has implemented a distributed multiplayer Pacman game that supports real-time synchronisation, scalable server deployment, and persistent scoring. We developed a system architecture that incorporates WebSockets for real-time communication, Redis for fast in-memory data handling, and MongoDB for long-term persistence. Players can interact across maze instances, with dynamic transitions, synchronised ghost behaviour, and shared dot collection logic.

Key achievements include a fully working multiplayer system with coordinated player movement, live leaderboard updates, maze sector transitions, and dropper functionality. We implemented a distributed maze management system that partitions maze sectors across multiple Docker-based server instances, improving scalability and fault isolation.

8 ALLOCATION OF WORK

See following page.

Team Members	Task
Sheena and Zemyna	Frontend Implementation
Thomas	Server Architecture and Multiplayer Functionality
Wen Geng	MongoDB Functionality
Complete Team	System Architecture Design
Thomas	Redis Chaching
Wen Geng	Dropper Implementation
Yongjia	Leader board Implementation and help with functional testing
Zemyna, Sheena, Thomas, and Wen Geng	Pair programming for error identification
Zemyna	Functional and Non-Functional Testing and Testing final report write up
Zemyna, Sheena and Thomas	Interim Reports
Sheena	Final Report
Complete Team	Bug Fixing